

Title: Libspace -A Robust Backend for Library Operations

TEAM ID: SWTID-2026-9662

TEAM SIZE:5

TEAM MEMBERS:

Team leader: T.Sharmila

Mail id: sharmisharmi6374@gmail.com

Team Members:

1.T.Priya

Mail id:priyathirupathi785@gmail.com

2.S.Keerthiga

Mail id:keerthisenthil1880@gmail.com

3.M.Kalai Nisha

Mail id:nishakala33@gmail.com

4.Y.Vaishnavi

Mail id :vaishnaviy931@gmail.com

Objective:

To develop a system that helps businesses track product and manage stock efficiently

LibSpace

Project Title

Libspace API - A Robust Backend for Library Operations.

1. Project Overview

LibSpace is a micro web-based library management system that provides secure user authentication and basic book management features. The application allows users to register, log in, log out, and delete their accounts. Once authenticated, users can perform CRUD (Create, Read, Update, Delete) operations on books, including adding new books, viewing the list of books, updating book details, and deleting books.

The main goal of LibSpace is to demonstrate authentication flow and CRUD operations while backend, and database functionalities in a structured manner.

2. Project Instructions – *LibSpace*

1. Register a new user account using the registration form.
2. Log in with the registered credentials to access the dashboard.
3. After successful login, perform book management operations:
 - Add a new book.
 - View the list of available books.
 - Get book by ID
 - Update book details.
 - Delete a book.
4. Users can log out securely after completing operations.
5. Users also have the option to delete their account permanently from the system.

These instructions describe how the system should be used once it is running.

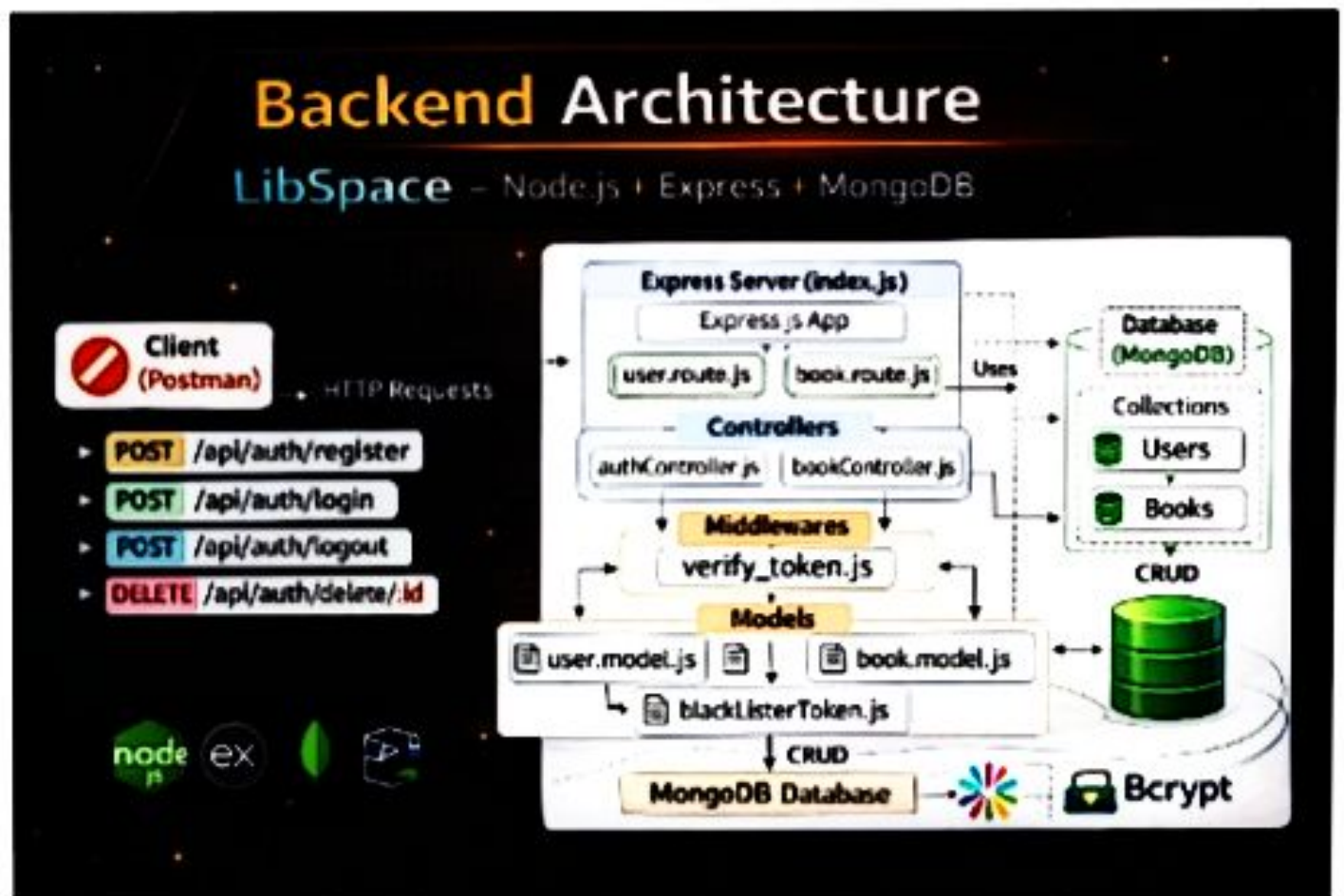
Pre-requisites

Before running or developing LibSpace, ensure the following:

- Basic knowledge of:
 - JavaScript
 - Backend concepts (e.g., Node.js / Express)
 - Database fundamentals (CRUD operations)
- Software Requirements:

- Node.js and npm installed
 - Database setup (MongoDB / MySQL)
 - Code editor (e.g., VS Code)
 - Web browser (Chrome/Firefox)
- Basic understanding of:
 - REST APIs
 - Authentication (sessions or JWT)
 - Environment variables configuration

3. Project Architecture



Technical Architecture

LibSpace follows a **Layered Backend Architecture** consisting of:

1. **API Layer (Routes)**

2. **Business Logic Layer (Controllers)**
3. **Data Access Layer (Models / Database)**

This structure ensures clean code organization, scalability, and maintainability.

1. API Layer (Routes):

The API layer handles incoming HTTP requests and defines application endpoints.

- Built using: Node.js + Express.js
- Responsible for:
 - Defining routes (e.g., `/register`, `/login`, `/books`)
 - Mapping routes to controllers
 - Handling HTTP methods (GET, POST, PUT, DELETE)
 - Applying middleware (authentication, validation)

Example Endpoints:

- POST → Register User
- POST → Login User
- DELETE → Delete User
- POST → Add Book
- GET → Fetch Books
- PUT → Update Book
- DELETE → Delete Book

2. Business Logic Layer (Controllers):

The controller layer contains the core application logic.

Responsibilities:

- Processing user registration and login
- Password hashing (e.g., bcrypt)
- Token generation (JWT) / session handling
- Implementing CRUD operations for books
- Validating request data
- Handling errors and sending responses

This layer acts as a bridge between routes and database.

3. Data Access Layer (Models / Database):

The data layer manages persistent storage.

- Database: MongoDB / MySQL
- Models define schema structure for:
 - Users (username, email, password)

- Books (title, author, published_date, etc.)

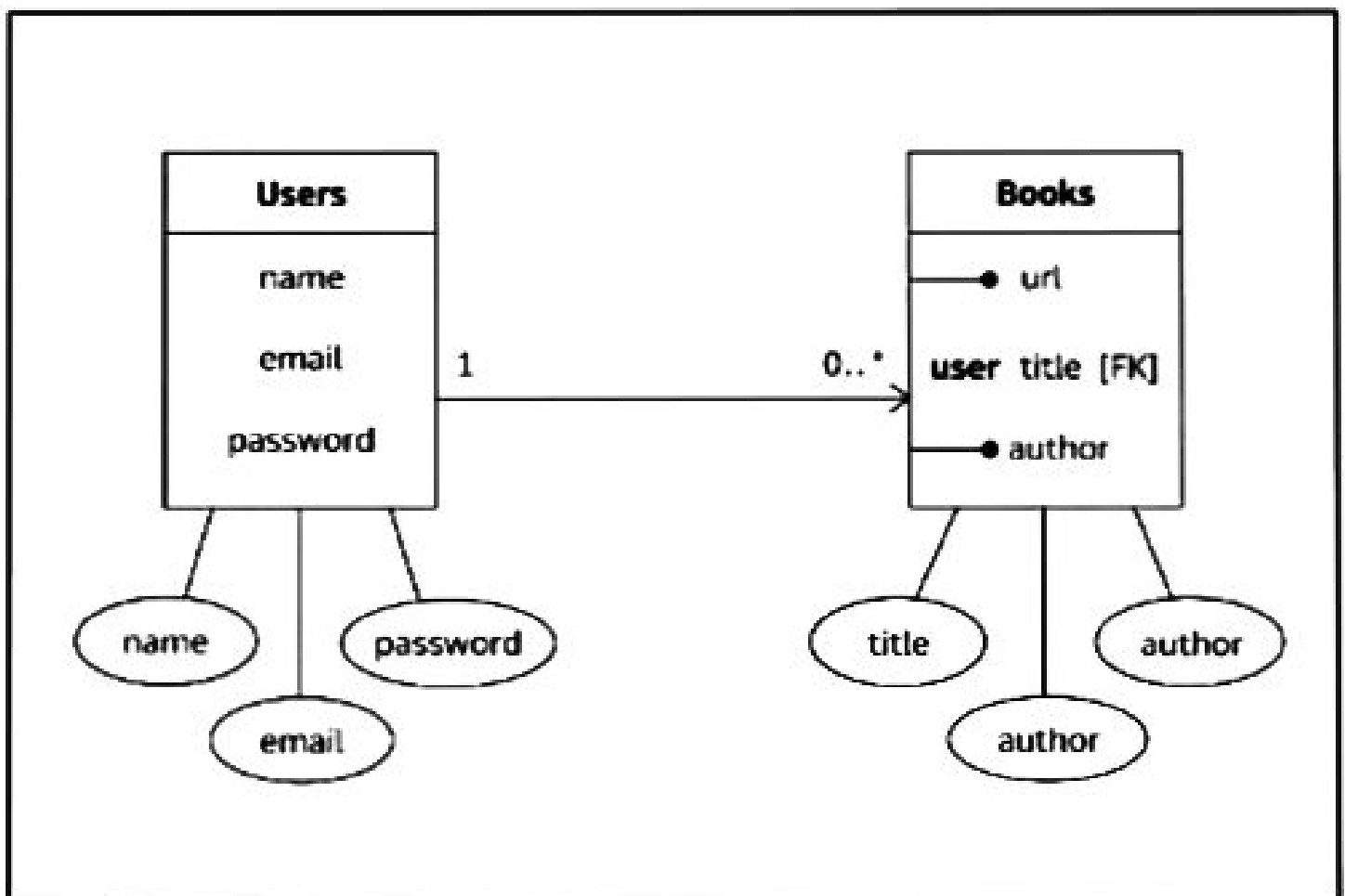
Responsibilities:

- Executing database queries
- Storing and retrieving data
- Maintaining data integrity

4. Request Flow:

1. Client sends HTTP request to API endpoint.
2. Route receives the request and forwards it to the controller.
3. Controller processes business logic.
4. Controller interacts with the database via models.
5. Database returns data to controller.
6. Controller sends structured JSON response back to client.

ER Diagram:



1. User Authentication

- User Registration (name, email, password)
- Secure password storage (hashed password)
- User Login with credential verification
- Token-based authentication (JWT)
- Delete User Account
- Automatic timestamps (createdAt, updatedAt)

2. Book Management (CRUD Operations)

Each book is linked to a specific user using a reference (`user: ObjectId`).

- Add a new book (url, title, author)
- Get all books of a specific user
- Get book by ID
- Update book details
- Delete a book
- Books are protected and accessible only to the authenticated user

3. User-Book Relationship

- One-to-Many Relationship:
 - One User → Can have multiple Books
 - Each Book → Belongs to one User (via ObjectId reference)
- Maintains data integrity using MongoDB references

4. Security & Validation

- Input validation for required fields
- Protected routes using authentication middleware
- Error handling for invalid requests
- Unauthorized access prevention

5. RESTful API Design

- Follows REST principles
- Uses standard HTTP methods:
 - POST
 - GET
 - PUT
 - DELETE
- Returns structured JSON responses

Roles and Responsibilities:

Requirement Analysis